

Fineract Analytics Platform

Design Report — an end-to-end, CDC-driven analytics engineering pipeline from the Apache Fineract REST API to analytics-ready marts and point-in-time-correct ML features.

```
Fineract API → PostgreSQL (OLTP) → Debezium / Kafka (CDC) → ClickHouse
                                     ↓
Airflow orchestrates → dbt: staging → intermediate → marts / ml
                                     ↓
                               BI · ML training · Grafana
```

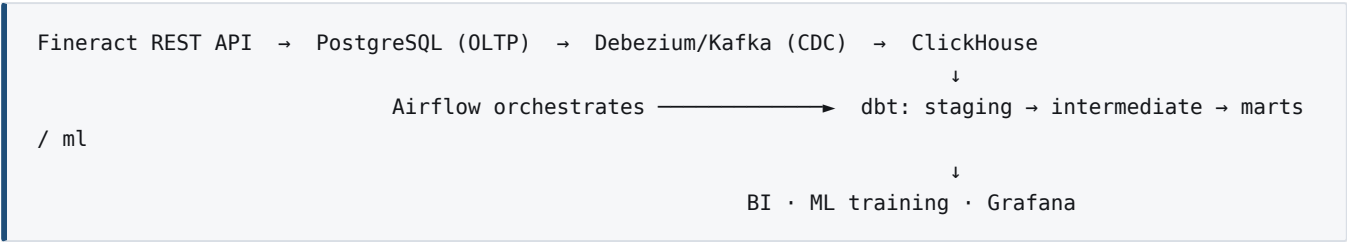
Stack Python 3.11 · PostgreSQL 16 · Debezium 2.7 · Kafka 3.7 (KRaft) · ClickHouse 24.8 · dbt 1.8 · Airflow 2.10 · Prometheus · Grafana · Docker Compose · GitHub Actions

Verified 22 dbt models built on a real ClickHouse engine · 133 tests passing · 5,510 rows ingested, 0 writes on re-run

Generated 11 August 2026

1. Overview

This is a production-shaped analytics platform for a microfinance institution running **Apache Fineract** as its core banking system. It takes data from the Fineract REST API through to analytics-ready marts and point-in-time-correct ML features, in near real time, with quality gates and observability at every stage.



The full architecture diagram is `docs/diagrams/architecture.mmd`; the change path for a single transaction is `docs/diagrams/cdc_sequence.mmd`.

One command starts everything: `make up`.

Why Apache Fineract as the source

The brief allows any public REST API. Fineract was chosen because it is the actual domain: it is the open-source core banking platform used by a large share of microfinance institutions, its API is genuinely awkward in instructive ways (integer-array dates, two different collection envelopes, child-only sub-resources, enum objects rather than scalars), and the resulting data model — clients, loans, a repayment ledger, delinquency — supports analytics that mean something. A weather API would have demonstrated the same plumbing against data with no consequences attached.

Three interchangeable source modes ship with the stack, selected by Docker Compose profile rather than by editing code:

Mode	Command	When
Self-hosted Fineract	<code>docker compose --profile fineract up -d</code>	Full fidelity, offline after image pull
Recorded-fixture mock	<code>docker compose --profile mock up -d</code>	CI, air-gapped review, deterministic tests
Public Apache demo	<code>set FINERACT_BASE_URL=https://demo.mifos.io/...</code>	Quick look, needs internet

The mock is not a shortcut. A test that depends on a shared, mutable, third-party demo database is not a test — it is a coin flip. CI runs the **real ingestion code** against the mock, which serves the same resource shapes including the `[yyyy, m, d]` dates and `pageItems` envelope, and can inject deterministic 503s to exercise the retry path.

2. Architecture

2.1 Component inventory

Layer	Technology	Role
Source	Apache Fineract	Core banking system of record
Ingestion	Python 3.11 service (<code>fineract_ingest</code>)	REST → Postgres, idempotent, validated, instrumented
OLTP	PostgreSQL 16 (<code>wal_level=logical</code>)	Landing mirror + pipeline control plane
CDC	Debezium 2.7 (pgoutput) + Kafka 3.7 (KRaft)	Change capture and transport
OLAP	ClickHouse 24.8	Kafka engine consumers, ReplacingMergeTree raw layer, warehouse
Transformation	dbt 1.8 + dbt-clickhouse	staging → intermediate → marts → ml
Orchestration	Airflow 2.10 (LocalExecutor)	3 DAGs, 43 tasks
Observability	Prometheus, Grafana, Pushgateway, statsd-exporter, custom exporter	Platform <i>and</i> data monitoring
CI/CD	GitHub Actions	9-job CI graph, gated CD, nightly quality run
Packaging	Docker Compose	17 services, 4 profiles, healthchecks throughout

2.2 Data flow, stage by stage

Stage 1 — REST → PostgreSQL. The ingestion service walks a declarative entity registry (`entities.py`), paginates each collection, maps the JSON to a flat row, validates it, and upserts on the Fineract natural key. The landed schema stays deliberately close to the API resource: no business logic runs during ingestion, so ingestion is replayable and every later transformation is version-controlled dbt rather than opaque Python.

Stage 2 — PostgreSQL → Kafka. Postgres runs with logical decoding. An explicitly declared publication, `fineract_cdc_pub`, defines the capture set in SQL that lives in git. Debezium streams it with the `pgoutput` plugin into `fineract.oltp.<table>` topics.

Stage 3 — Kafka → ClickHouse. ClickHouse Kafka engine tables consume the topics; materialized views convert wire types to analytical types and write into `ReplacingMergeTree(_version, _is_deleted)` tables in `fineract_raw`. Typical end-to-end latency, Postgres commit to queryable row, is **1-3 seconds**.

Stage 4 — ClickHouse → marts. dbt builds 22 models across four layers. Airflow runs the layers in sequence so that a marts failure never looks like a staging failure.

Stage 5 — quality and publication. dbt tests, then a warehouse-level quality gate, then dbt artefacts are loaded into `fineract_ops` so test history is queryable rather than buried in task logs.

2.3 The decision that shapes everything else: the CDC gate

Ingestion writes to Postgres. dbt reads from ClickHouse. The CDC stream between them is asynchronous and outside Airflow's control. Running dbt immediately after ingestion would transform a warehouse that has not yet received the rows, **succeed**, and publish stale numbers. That is the worst failure mode a pipeline can have, because nothing is red.

So the pipeline has an explicit gate — `CDCCaughtUpSensor` — between ingestion and transformation, and it requires **two independent conditions**:

1. **Row parity.** ClickHouse live-key counts match Postgres counts per entity. Catches events dropped entirely.
2. **Freshness.** The newest source commit visible in ClickHouse is under five minutes old. Catches a stream that is merely slow.

Neither alone is sufficient: an idle stream trivially satisfies row parity, and a stream delivering some events while silently dropping others still looks perfectly fresh.

3. Data model

The ERD is `docs/diagrams/data_model.mmd`.

3.1 Layering

Layer	Database	Materialisation	Contract
<code>fineract_raw</code>	<code>fineract_raw</code>	ReplacingMergeTree, fed by MVs	Append-only, at-least-once, uncollapsed. Only staging may read it.
staging	<code>fineract_staging</code>	table	One current row per key, typed, renamed. No joins, no business rules.
intermediate	<code>fineract_intermediate</code>	table	Reusable business logic. Not exposed to BI.
marts	<code>fineract_marts</code>	table / incremental	Analytics-ready facts and dimensions.
ml	<code>fineract_ml</code>	table	Feature tables with explicit temporal semantics.
<code>fineract_ops</code>	<code>fineract_ops</code>	tables + views	Pipeline telemetry, kept apart from the data itself.

One database per layer rather than name prefixes: grants, retention and `DROP DATABASE` blast radius all follow the layer boundary.

The staging layer contains exactly one piece of business judgement, and it is there on purpose. Fineract reports delinquency two ways (`delinquent.pastDueDays` and `summary.overdueSinceDate`) and they disagree when the delinquency-bucket configuration changes. Every consumer must use the same reconciliation, so it happens once, at the boundary, with the rule written down.

3.2 ClickHouse-specific design choices

Engine — `ReplacingMergeTree(_version, _is_deleted)` in the raw layer. CDC delivery is at-least-once and out of order, so the raw layer must be able to collapse N versions of a key. `_version` is the **source commit time** (`__source_ts_ms`), never the Debezium read time: if the connector falls behind and catches up, read-time versioning lets an older row overwrite a newer one. `_is_deleted` turns a delete into a reconcilable fact rather than a ghost row. *This is verified, not asserted* — see §8.

Sorting keys are chosen from real filter patterns, not defaults. In ClickHouse the `ORDER BY` is the primary index. `fct_loan` is ordered `(disbursed_on_date, office_id, product_id, loan_id)` because that is the order the dashboards filter in: a period, then a branch, then a product. `fct_loan_transaction` is ordered `(transaction_date, office_id, loan_id, transaction_id)` for the same reason. Getting this wrong is the difference between a granule scan and a full scan.

Partitioning is applied only where it prunes. Monthly partitions on the two date-heavy fact tables (`toYYYYMM(transaction_date)`, `toYYYYMM(disbursed_on_date)`); none on dimension-sized tables, where partitioning would multiply parts and slow merges for zero benefit. Partitioning `loan_transactions` by transaction date is safe *because a Fineract transaction's date is immutable* — a correction is a new reversal transaction, never a date edit — so no row can ever need to move partitions, which would break dedup.

A bloom-filter skip index on `loan_transactions.loan_id`. The leading `transaction_date` in the sort key cannot prune "show me this borrower's ledger", which is the single most common operational query. The skip index restores it.

Codecs. `Delta + ZSTD` on monotonic columns (ids, versions, LSNs, dates), `LowCardinality` on enum-like strings. Roughly 4–6× smaller than defaults on the fact table.

Money is `Decimal(19,6)` end to end. Debezium is configured with `decimal.handling.mode=string` so full precision survives the wire, and ClickHouse casts with `toDecimal64OrNull`. Never `Float64`: a portfolio total that disagrees with the core banking system by 0.0000001 per row is a reconciliation meeting nobody enjoys.

Materialized views do all the type conversion. A Kafka engine table that cannot parse a message stalls the whole partition, so the consumer is made maximally forgiving (`input_format_skip_unknown_fields`, `kafka_handle_error_mode='stream'`) and the strictness moves one hop downstream where a failure is recoverable. Poison messages are routed to `fineract_raw.cdc_errors` with the raw bytes, and alert.

`argMax` where it matters, `FINAL` where it does not. Dimension-sized staging models use `FINAL` — correct at any merge state, one line at the call site. The transaction fact uses explicit `argMax`, because `FINAL` applied *after* an incremental `WHERE` collapses only the rows inside the window, and two versions straddling the boundary produce a partially-collapsed row: silently wrong and very hard to spot. (ADR-0003.)

3.3 The ML layer, and why it is two tables

`ml_loan_default_features` is a **training** table: every feature is computed strictly from information that existed *before* the loan's disbursement date. Prior-loan features join on `prev.disbursed_on_date < this.disbursed_on_date`; prior-repayment features only read ledger rows before the observation date; age and tenure are re-derived as-of origination rather than taken from the current dimension.

Deliberately absent: the loan's own repayment behaviour, current balances, current delinquency, `client_segment`. Every one of those is post-origination and would leak the label. A model trained on them scores about 0.95 AUC offline and is worthless in production — the single most common failure in credit-risk feature engineering. A singular dbt test, `assert_ml_features_are_point_in_time`, encodes the invariant so a refactor cannot quietly break it.

`ml_client_scoring_features` is the **serving** table: current-state features for scoring a client today. It legitimately contains what would be leakage in training. Keeping them as two tables with different rules is what stops that distinction from being lost, and the shared feature names are identical on purpose so one model applies to both.

`split_hint` is time-based, not random: a random split leaks the future into training for a temporally ordered process. `is_label_mature` excludes loans too young to have shown default behaviour — without it the model learns "new loans never default".

4. Data quality and testing

Quality is enforced at four distinct points, because each catches a different class of failure.

1. Row level, at ingestion. A record that fails validation is *quarantined* in `meta.ingestion_reject` with its raw payload and the reason — not dropped, not fatal. One malformed loan must never fail a 40,000-row batch, and must never vanish silently either. The run fails only if the reject *ratio* crosses a threshold, because a spike in rejects means the source contract changed.

2. Batch level, before commit. Declarative expectations live next to each entity in the registry (not-null, unique, non-negative, range, row-count-min) with `error` / `warn` severities. Blocking failures abort the load *before it is committed*, so a bad batch never reaches the CDC stream. Failing closed is the right default for financial data. Results land in `meta.data_quality_result` and become Prometheus metrics.

3. Warehouse level, via dbt. Schema tests on every model, plus five singular tests that encode domain invariants:

Test	What it protects
<code>assert_loan_ledger_reconciles</code>	The loan summary equals the sum of its ledger. The single most valuable test here — a CDC event lost between Postgres and ClickHouse shows up here and essentially nowhere else, because the loan row still looks perfectly valid on its own.
<code>assert_portfolio_balances_are_sane</code>	Accounting identities: outstanding $\leq$ disbursed, no negative balances, overdue $\leq$ outstanding, not both active and closed.
<code>assert_par_bucket_matches_days_past_due</code>	The PAR band and the day count cannot drift apart.
<code>assert_ml_features_are_point_in_time</code>	No label leakage in the training set.
<code>assert_no_future_dated_transactions</code>	Future-dated repayments inflate collections and deflate PAR — the direction of error nobody notices until an audit.

4. Pipeline level, after the marts are built. `DataQualityGateOperator` runs warehouse-wide assertions as the last task. A failure is loud *and* the marts exist, so the on-call engineer inspects what was actually produced instead of guessing.

Unit and integration tests. 105 tests: parsing (Fineract's four date encodings, decimal precision, hash stability), the HTTP client against a real socket (pagination termination, deterministic retry injection, rate limiting), the registry and mappers, pipeline mapping and dedup logic, and — against a real Postgres — idempotency, churn suppression, transactional watermarks, quarantine, and CDC readiness (publication coverage, `wal_level`, replica identity).

Two validators that execute real SQL with no containers. `scripts/validate_clickhouse_sql.py` and `scripts/validate_dbt_sql.py` run the ClickHouse DDL and build the entire dbt DAG on an *embedded* ClickHouse engine (chdb), then run the data tests. `dbt parse` only checks that Jinja renders; `dbt compile` needs a warehouse. Neither tells you whether the SQL is valid ClickHouse until something is running — which in practice means a broken model is found by the nightly run rather than by the pull request. These close that gap in about two seconds, on every push.

5. Orchestration

Three DAGs, 43 tasks.

`fineract_analytics_pipeline` (every 4 hours) — preflight → ingest → CDC gate → transform → test → publish → quality gate.

- *One DAG, not four.* Ingestion, CDC verification, transformation and quality are one dependency chain with one SLA. Splitting them across DAGs linked by `ExternalTaskSensor` is the usual approach and usually a mistake: it hides the real dependency and turns "is today's data good?" into a question about four separate run histories.
- *Ingestion tasks are generated from the entity registry*, so the DAG cannot drift from what the ingestion service actually loads — asserted by a test.
- *Sequential ingestion within the group.* Parent/child entities have a real dependency, and eight parallel crawlers against a live core banking system is precisely what the client-side rate limiter exists to prevent.
- *Every 4 hours, not every 15 minutes.* Branches post transactions in daylight hours; a 15-minute cadence would mostly crawl an unchanged API and hold a replication slot open for nothing. CDC already delivers the real-time path.
- `catchup=False`, `max_active_runs=1`. This pipeline transforms current state; two concurrent runs would race on the same relations.

`platform_maintenance` (hourly) — the housekeeping that quietly breaks a working CDC pipeline: heartbeat, replication-slot lag, ClickHouse merge pressure, retention, and a report of open rejects. An idle capture set means the slot's confirmed LSN never advances and Postgres retains WAL until the disk fills; this is the most common way a working CDC pipeline takes down its source database at 3am.

`fineract_backfill` (manual only) — re-ingestion and full rebuild, behind a typed confirmation parameter, with before/after row-count verification. Separate from the scheduled pipeline so a mistyped parameter on a scheduled run cannot rebuild the warehouse.

6. Observability

Four questions, and a metric set built to answer each:

Question	Signals
Did it run?	<code>fineract_ingestion_last_success_timestamp_seconds{entity}</code> , Airflow StatsD via statsd-exporter
Did it work?	<code>..._rows_last_run</code> , <code>..._run_status</code> , <code>fineract_pipeline_failed_tasks</code>
Was the data good?	<code>..._rejected_rows_total</code> , <code>fineract_dbt_test_failures{layer}</code> , <code>fineract_cdc_parse_errors_total</code>
Was it fast, and is it current?	<code>fineract_cdc_freshness_seconds{table}</code> , <code>..._lag_p50/p95</code> , <code>fineract_cdc_replication_slot_lag_bytes</code>

Freshness and lag are separate metrics on purpose. Freshness is how old the newest row is, measured against source commit time — what a business user means by "how current is the data". Lag is commit-to-queryable — the pipeline's own latency. A quiet Sunday makes freshness look terrible while lag is perfect; alerting on freshness alone pages you for an idle source.

The metric that catches what nothing else can. `fineract_reconciliation_row_delta{entity}` compares Postgres row counts with ClickHouse live-key counts. Every per-component health check can be green while CDC silently drops events; this is the one signal that sees it.

Batch jobs push, long-lived services are scraped. Ingestion exits before Prometheus could reach it, so it pushes to a Pushgateway with `honor_labels: true`. Everything else is scraped: the custom pipeline exporter (:9105), ClickHouse's native endpoint (:9363), `postgres_exporter`, `kafka_exporter`, a JMX agent on Kafka Connect exposing Debezium's `MillisecondsBehindSource`, `statsd-exporter` for Airflow, and Fineract's own Spring Boot actuator — source latency is a pipeline concern.

18 alert rules with `for:` durations and a `runbook` annotation on each, plus recording rules for the expensive repeated expressions. Three Grafana dashboards: pipeline health, CDC and data quality, portfolio overview. Alert-by-alert procedures are in [RUNBOOK.md](#).

7. CI/CD

Nine jobs in a real dependency graph, not a flat list:

```
lint → unit-tests → sql-validation → dbt-checks → integration-tests · connector-config-validation · compose-validation → build-images → ci-summary
```

Worth singling out:

- sql-validation** executes every ClickHouse DDL statement and builds the entire dbt DAG in-process via `chdb`. No containers, seconds, every push.
- dbt-checks** includes a governance gate that reads `manifest.json` and fails if a model has no description or a staging model has no test. A real gate, not a rubber stamp.
- connector-config-validation** asserts the Debezium config still has `plugin.name=pgoutput`, `decimal.handling.mode=string`, a heartbeat interval, and the unwrap SMT with

`delete.handling.mode=rewrite`. A typo in any of those fails *silently at runtime*, which is exactly why it is a build gate.

- `integration-tests` stands up real Postgres and ClickHouse services, applies the DDL, runs the real ingestion against the mock source, and asserts rows landed.
 - **CD** is tag-triggered with a GitHub Environments approval gate, uses the dbt slim-CI pattern (`state:modified+ --defer --state`), and runs a post-deploy smoke check against the exporter's `/metrics` asserting freshness before declaring success.
-

8. Verification — what was actually proven, and how

Every claim below was executed, not asserted. The container registries in the build environment were unreachable, so the stack could not be booted there; instead each layer was verified against real engines installed natively.

Claim	How it was verified	Result
OLTP DDL is valid and CDC-ready	Applied all four init scripts to real PostgreSQL 16	8 tables, publication covering 9 relations, <code>wal_level=logical</code>
Ingestion works end to end	Real ingestion against the mock API into real Postgres	5,510 rows across 8 entities, 0 rejects
Idempotency and churn suppression	Immediately re-ran the identical load	5,510 read, 0 inserted, 0 updated, 5,510 unchanged — no WAL, therefore no CDC events
ClickHouse DDL is valid	All 53 statements on a real ClickHouse engine (chdb 26.5)	All executed
Decimal precision survives CDC	Round-tripped <code>120345.678900</code> as a Debezium string	Exact, typed <code>Decimal(19,6)</code>
Date conversion	Debezium <code>Int32</code> days → ClickHouse <code>Date</code>	Correct
Delete reconciliation	Emitted <code>__op='d'</code>	<code>_is_deleted</code> resolved to 1
Out-of-order guard	Injected an event with an <i>older</i> source commit time after a newer one	Stale value did not win — version semantics correct
Poison-message quarantine	Injected an unparseable message	Routed to <code>cdc_errors</code> , consumer unaffected
dbt DAG is valid ClickHouse	Built all 22 models on the real engine	All built
Data tests pass	All schema tests + 5 singular tests on the built DAG	All passed
Airflow DAGs are correct	18 integrity tests against real Airflow 2.10.5	All passed; 43 tasks, no cycles, CDC gate correctly positioned
Test suite	105 unit + integration tests	All passed
Compose is valid	<code>docker compose config</code>	Valid; 17 services, 4 profiles

Two real bugs were caught by this process and fixed, both of which would have failed at runtime and neither of which any amount of review would reliably have caught:

- Ambiguous unaliased columns.** In ClickHouse, a `SELECT c.client_id` that is ambiguous across joined relations produces a column literally named `c.client_id`. Everything downstream then fails with a misleading "correlated subqueries are not supported" error. An automated check for this is now part of the validator.
- Alias shadowing in aggregates.** `sumIf(total_outstanding, ...) AS total_outstanding` followed by another `sumIf(total_outstanding, ...)` resolves the second reference to the *alias* — an aggregate inside an aggregate. Fixed by qualifying every source column, and documented in the model.

9. Scaling and extension

Where this design holds, where it bends, and what to change.

9.1 What the current shape supports

Single-node, laptop-class: roughly **10 million loan transactions** and a few hundred change events per second before anything needs to move. The constraint is ClickHouse merge throughput on one node, not ingest.

9.2 10× — hundreds of millions of rows

- **ClickHouse → 3-node cluster.** Switch the engines to `ReplicatedReplacingMergeTree`, add `cluster` to the dbt profile (the `prod` target already carries the setting), shard the transaction fact by `cityHash64(loan_id)` so a borrower's ledger stays on one shard, and keep the dimensions as fully-replicated tables so joins stay local.
- **Kafka → partition by key, scale consumers.** Topics are already partitioned by primary key, so ordering per key survives. Raise `kafka_num_consumers` on the transaction table and add brokers.
- **Ingestion → parallel per entity.** The entity registry already makes each entity an independent task; move from sequential to a pool-bounded parallel group, with the rate limiter budget divided between workers so the source system's load is unchanged.
- **Airflow → CeleryExecutor or Kubernetes.** The `DbtOperator` shells out to a CLI, so moving to `KubernetesPodOperator` is a config change, not a rewrite. That seam was chosen for this reason.

9.3 100× — the shape changes

- **Stop crawling the API for facts.** Above a certain volume the REST crawl becomes the bottleneck and a burden on the source. Fineract emits business events; subscribe to those for the transaction stream and keep the REST crawl for dimensions only.
- **Tiered storage.** ClickHouse `TTL ... TO DISK/VOLUME 's3'` moves cold partitions to object storage. Partitioning by month already makes this a one-line policy change.
- **Aggregating engines.** Replace the daily snapshot's full recompute with `AggregatingMergeTree` and incremental materialized views on the hot aggregates.
- **A real feature store.** The ML tables are currently dbt models. At scale, point-in-time correctness for online serving needs an actual feature store (Feast on ClickHouse); the offline definitions here map onto it directly because the temporal semantics are already explicit.

9.4 Extension points designed in

- **A new source entity** is one `EntitySpec` — the client, loader, validator, CLI, metrics and Airflow tasks all derive from the registry.
- **A new source system** implements the same `EntitySpec` contract; the CDC, warehouse and orchestration layers do not change.
- **A new mart** is a dbt model plus a YAML entry; the governance gate makes the description and tests mandatory.
- **Multi-tenancy** — Fineract is multi-tenant and the tenant header is already configuration. The path is a tenant column through the landing tables and a tenant dimension, not a second pipeline.

9.5 Known limitations, stated plainly

- **dim_client** is **Type 1, not Type 2**. History exists in `fineract_raw.cdc_audit` and in the uncollapsed raw layer; a third representation would be a third source of truth. If point-in-time client attributes become a hard requirement, the right move is a dbt snapshot over the staging model, not a hand-rolled SCD.
- **The daily portfolio snapshot accumulates forward**. It cannot be rebuilt for days before it started running, because Fineract reports only current balances. This is a property of the source, not a defect, but it means the snapshot table must be backed up like data, not treated as derived.
- **The prior-loan ML join is an inequality self-join**. Fine at demo scale; at millions of loans it wants an `ASOF JOIN` or a windowed pre-aggregation.
- **Single-node everything**. No replication, no HA. Deliberate for a reviewable local stack; \$9.2 is the path out.

10. Security posture

Local defaults are development credentials and nothing here is a secret. For a real deployment:

- **Secrets** move to the platform's secret manager. The Debezium connector config already reads `${env:...}` rather than embedding credentials, and the ingestion service is entirely environment-driven, so this is configuration, not code.
- **Least privilege is already modelled**. `app_ingest` has DML only; `debezium` has `REPLICATION` plus `SELECT`; `analyst_ro` is read-only; ClickHouse has separate `analytics_etl` and read-only `bi_reader` profiles with different resource limits, so a runaway dashboard query cannot starve the merges that keep the raw layer collapsed.
- **TLS** everywhere (`FINERACT_VERIFY_SSL=true` with a mounted CA bundle, ClickHouse `secure: true` in the prod target, SASL/TLS on Kafka).
- **PII**. The client table carries names, phone numbers, email and date of birth. In a regulated deployment these should be tokenised at ingestion, with the marts carrying only the derived bands (`age_band` , `tenure_band`) that the analytics actually use — which is why those bands exist as columns rather than being computed in BI.

11. Trade-offs I would revisit

Honest notes, rather than a list of things that went well.

ClickHouse Kafka engine over a sink connector. Fewer moving parts and back-pressure visible in `system.kafka_consumers` rather than buried in Connect logs. The cost is that the consumer schema now lives in ClickHouse DDL; mitigated with `input_format_skip_unknown_fields`, but at a larger organisation, where the Connect cluster is already an operated platform, `clickhouse-kafka-connect` would be the better fit. (ADR-0001.)

Hash-based change detection. Cheap and effective — verified at 0 writes on a 5,510-row unchanged re-run. The cost is a SHA-256 per record and the risk that a mapper change silently rewrites every row once. (ADR-0002.)

dbt inside the Airflow image. One fewer container for a local stack; in production it should be a separate image so dbt and Airflow can be upgraded independently. The CLI seam makes that swap cheap, and that is why the operator shells out rather than importing dbt's Python API — which is explicitly not a stable interface.

Ingestion is single-threaded per entity. Correct for a live core banking system and simple to reason about, but it means the `loan_transactions` crawl is $O(\text{loans})$ sequential API calls. At scale this is the first thing that has to change, and §9.3 says how.

LocalExecutor. Right for a reviewable stack, wrong for production.

12. Repository map

— docker-compose.yml	17 services, 4 profiles, healthchecks throughout
— Makefile	28 self-documented targets; `make help`
— ingestion/	Fineract REST → Postgres (package + Dockerfile + tests)
— platform/	
— postgres/	OLTP schema, control plane, CDC publication, tuned conf
— clickhouse/	Kafka sources, raw tables, MVs, ops views, profiles
— kafka/	JMX → Prometheus mapping for Debezium
— cdc/	Debezium connector config + idempotent registration
— transform/fineract_analytics/	dbt: 22 models, 5 singular tests, macros, seeds
— orchestration/	3 Airflow DAGs, custom hooks/sensors/operators, tests
— observability/	Prometheus rules, Grafana dashboards, custom exporter
— scripts/	The two embedded-engine SQL validators
— tests/	Unit + integration suites
— .github/workflows/	CI, CD, nightly quality
— docs/	This report, RUNBOOK, ADRs, diagrams

Every number in §8 came from an actual execution, and the commands that produced them are in `RUNBOOK.md` and the two validator scripts.